

HelixTrack

HelixTrack

JIRA альтэрнатыва для свабоднага свету.

Зводка

HelixTrack — гэта сучасная адкрытая крыніца-адкрытая альтэрнатыва JIRA (а праз пашырэнне Documents — і Confluence), шматплатформавая сістэма кіравання праектамі і адсочвання праблем, пабудаваная на мікрасэрвіснай архітэктурцы Go з натыўнымі кліентамі для вэб, дэсктопа і мабільных прылад.

Кароткае апісанне

Альтэрнатыва JIRA/Confluence з адкрытым зыходным кодам. Мікрасэрвісны бэкэнд на Go («HelixTrack Core») прапануе адзіны REST API для кіравання праектамі і адсочвання праблем, а таксама працоўную прастору тыпу Confluence, якая падаецца натыўным кліентам для вэб, дэсктопа, Android і iOS праз HTTP/3 QUIC.

Падрабязнае апісанне

HelixTrack — гэта платформа кіравання праектамі і адсочвання праблем з адкрытым зыходным кодам, якая пазіцыянуецца як альтэрнатыва для свабоднага свету замест JIRA і Confluence — поўная замена двум прадуктам, у якіх зачыненыя большасць інжынерных арганізацый, перабудаваная як праграмае забеспячэнне, якое належыць вам і можа працаваць дзе заўгодна. Яе аснова — **HelixTrack Core**, мікрасэрвіс на REST API, напісаны на Go з выкарыстаннем фрэймворка Gin. Ён прапануе поўнае адсочванне праблем, agile/scrum-дошкі, кіраванне камандамі і іерархічную сістэму дазволаў, рэалізацыя якой можа быць заменена паміж лакальным рухавіком у працэсе і HTTP-

сэрвісам — так адна і тая ж мадэль аўтарызацыі маштабуюцца ад адзінага ноўтбука да размеркаванага кластара без змен у прыкладным кодзе. Замест таго, каб распаўсюджваць REST паверхню па дзясятках маршрутаў, Core сканцэнтравана на адзіным эндпойнце /do з маршрутызацыяй па дзеяннях і адзіным фарматам запыту/адказу (action/jwt/object/data на ўваходзе, errorCode/errorMessage/data на выхадзе): усе кліенты выкарыстоўваюць адзін і той жа мінімальны кантракт, а дадаванне новай функцыі азначае дадаванне дзеяння, а не новага URL, які трэба дакументаваць, абараняць і версіяваць. Core інтэгруецца з асобнымі сэрвісамі аўтэнтыфікацыі, дазваляе і лакалізацыі, якія звязаны праз HTTP/3 QUIC і могуць працаваць на асобных машынах ці кластарах або адключацца ў тэставых канфігурацыях. Данія захоўваюцца ў SQLite для распрацоўкі без папярэдняй наладкі і ў PostgreSQL у прадукцыйным асяроддзі, шыфруюцца на дыску з дапамогай SQLCipher (AES-256), каб канфідэнцыйныя данія праекта абараняліся па змоўчванню, а не як дадатковая мера. Пашырэнне **Documents V2** ператварае трэкер у поўнатэлеую платформу ведаў: працоўная прастора тыпу Confluence з прасторамі, старонкамі, кантролем версій, шаблонамі, рэжымам рэальнага часу WebSocket і аналітыкай — вікі і трэкер канчаткова аб'ядноўваюцца ў адным бэкэндзе замест двух злепленых прадуктаў. Вакол Core існуюць некалькі кліенцкіх праграм: Angular-вэб-кліент, дэсктоп-кліент на Tauri + Angular, натывныя дадаткі для Android (Kotlin) і iOS (Swift), а таксама кліенты для HarmonyOS і Aurora OS і застаўка — усе яны звязаны з адным і тым жа бэкэндам і аўтаматычна знаходзяць яго ў лакальнай сетцы праз UDP-шыроканадзённую перадачу, таму новы кліент знаходзіць сервер без ручной наладкі. Кліенцкія праграмы падтрымліваюцца ў асобных закрытых рэпазіторыях і падаюцца тут толькі на ўзроўні прадукту.

Змест

Чаму мы яго стварылі

Каб даць камандам сапраўдную адкрытую, самастойна разгортвальную альтэрнатыву стэку JIRA + Confluence — «для свабоднага свету» — без прывязкі да пастаўшчыка, аб'яднаўшы карпаратыўнае адсочванне, дакументы і супрацоўніцтва пад адной адкрытай ліцэнзіяй.

Чаму гэта змяняе правілы гульні

Гэта аб'ядноўвае два магутныя камерцыйныя прадукты — адсочванне заданняў і стэк вікі/дакументаў — у адну адкрытую, высокапрадукцыйную, самастойна разгортвальную

платформу і дадае да іх нешта, чаго ніколі не прапаноўвалі існуючыя рашэнні: сапраўдныя мультыплатформенныя *натывныя* кліенты (web, desktop, Android, iOS, а таксама HarmonyOS і Aurora OS), якія працуюць на аснове адзінага кантракту бэкенда. Галоўны прарыў — гэта валоданне без кампрамісаў. Дызайн HTTP/3-паўсюднага, цалкам дэкапліраванага мікрасэрвіснага архітэктурны разам з шыфраваннем AES-256 SQLCipher на спакоі дае прадукцыйнасць і ўзровень бяспекі, які звычайна даступны толькі ў прапрыетарных SaaS-рашэннях, але ў сістэме, якую вы разгортваеце самі — без абмежаванняў на колькасць карыстальнікаў, без прывязкі да пастаўшчыка, без выхаду вашых даных за межы інфраструктуры. Каманды атрымліваюць досвед выкарыстання JIRA + Confluence, да якога яны звыклі, на ўласным абсталяванні і пад адной адкрытай ліцэнзіяй.

Што новага

- Адзіны дзеянне-арыентаваны /do API — адзін канцавы пункт, адзін канверт, маршрутызацыя па дзеяннях. Новыя магчымасці з'яўляюцца як новыя дзеянні, а не новыя URL-адрасы, што скарачае паверхню атакі, кліенцкі код і нагрузку на дакументацыю да адзінага кантракту, які падзяляюць усе платформы.
- HTTP/3 QUIC як *стандартны* транспарт паміж сэрвісамі — сучасная нізкалатэнтная, устойлівая да разрываў сувязі сетка паміж сэрвісамі з самага пачатку, а не дададзеная пазней.
- Рухавік дазваляў, які можна замяніць паміж лакальнай рэалізацыяй у працэсе і сэрвісам на базе HTTP, разам з магчымасцю незалежнага разгортвання сэрвісаў аўтэнтыфікацыі, дазваляў і лакалізацыі — аднолькавая мадэль аўтарызацыі незалежна ад таго, ці запускаяе вы адзін працэс ці кластар.
- Ізаляцыя даных паміж прасторамі праз сцяг --space-root: кожны праект атрымлівае ўласную ізаляваную базу даных і сховішча рэсурсаў, таму кліенты і праекты падзяляюцца на ўзроўні захоўвання, а не праз фільтры запытаў.
- Шыфраванне AES-256 SQLCipher на спакоі — канфідэнцыйныя даныя праекта абаронены на дыску празрыста і па змаўчанні.
- Аўтаматычнае выяўленне кліентам сервера праз UDP-шыроканакіраваную трансляцыю ў лакальных сетках — кліент знаходзіць Core без ручной канфігурацыі.
- Дакументы V2, сапраўдная «альтэрнатыва Confluence», з паралельным рэдагаваннем з аптымістычным блакаваннем, выяўленнем канфліктаў і поўнай гісторыяй зменаў —

сапраўдная калектыўная праца з дакументамі, якія жывуць за тым жа бэкендам, што і трэкер.

Найбольшыя тэхнічныя выклікі і як мы іх пераадолелі

- **Шэсць кліенцкіх платформ, адзін бэкенд, нулявы дрэйф кантракту.** Падтрымка Web/Angular, Desktop/Tauri, Android/Kotlin, iOS/Swift, HarmonyOS і Aurora звычайна азначае шэсць разбежных API-інтэграцый, якія паступова губляюць сінхранізацыю. Мы знізілі гэты рызык, зрабіўшы адзіны дзеянне-арыентаваны /do API і яго нязменны канверт *адзіным* кантрактам — усе кліенты накіраваны на яго аднолькава — і дадалі UDP-шыроканакіраванае выяўленне сэрвісаў зверху, каб кліенты знаходзілі Core у сетцы без ручной канфігурацыі канцавых пунктаў.
- **Дэкапліраванне сэрвісаў без страты прадукцыйнасці.** Раздзяленне аўтэнтыфікацыі, дазваляў і лакалізацыі на незалежна разгортвальныя сэрвісы звычайна дадае сеткавы хоп на кожны выклік. Мы выкарысталі HTTP/3 QUIC для ўсіх міжсэрвісных выклікаў, каб захаваць гэтыя хопы хуткімі і ўстойлівымі да разрываў сувязі, а таксама зрабілі кожны сэрвіс незалежна запусчальным — нават цалкам адключаным у тэставых канфігурацыях — такім чынам дэкапліраванне становіцца выбарам разгортвання, а не нязменнай нагрузкай.
- **Супрацоўніцтва ўзроўню Confluence без страты зменаў.** Рэдагаванне ў рэжыме рэальнага часу некалькімі аўтарамі спараджае канфлікты зменаў. Дакументы V2 вырашаюць гэтую праблему праз прасторы/старонкі/версіраванне з аптымістычным блакаваннем, а таксама празрыстае выяўленне канфліктаў, поўную гісторыю зменаў для вяртання і сінхранізацыю WebSocket у рэжыме рэальнага часу — супрацоўніцтва застаецца кансістэнтным замест таго, каб ціха перазапісваць змены.
- **Шыфраванне на спакоі без страты прапускнай здольнасці.** AES-256 SQLCipher абараняе даныя на дыску, але дадае нагрузку на кожны запыт; мы кампенсавалі гэта шматслойным кэшаваннем (LRU у памяці перад Redis у сэрвісе лакалізацыі), каб гарачыя шляхі, як шматмоўны пошук, заставаліся хуткімі, пакуль даныя застаюцца зашыфраванымі.

Змесціва

Тэхналагічны стэк

- **Go + Gin** — абраны для стварэння HTTP-сэрвісаў з высокай прапускнунай здольнасцю і нізкай затрымкай, якія разгортваюцца ў выглядзе адзінага бінарнага файла; уключае REST API ядра, яго JWT/CORS-прамежковы слой і маршрутызатар /do, які абслугоўвае ўсю сістэму.
- **HTTP/3 QUIC** — абраны для перадачы даных паміж ядром і сэрвісамі аўтэнтыфікацыі/дазваў/лакалізацыі, паколькі мультыплексная канструкцыя QUIC з міграцыяй злучэнняў скарачае затрымкі ў «хвасце» і застаецца стабільнай нават пры нястабільных злучэннях, дзе TCP губляе прадукцыйнасць.
- **PostgreSQL (прадукцыйнае асяроддзе) / SQLite (распрацоўка)** — адзіная рэляцыйная мадэль для вялікай схемы адсочвання і дакументаў, якая падтрымліваецца абодвума рухавікамі: SQLite забяспечвае распрацоўку без папярэдніх налад і захоўванне даных у файлах, у той час як Postgres у прадукцыйным асяроддзі маштабуецца праз асобны профіль production у Docker Compose.
- **SQLCipher (AES-256)** — абраны для празрыстага шыфравання даных на ўзроўні базы, што дазваляе абараняць канфідэнцыйную інфармацыю праектаў без змен у прыкладным кодзе і без змянення спосабу напісання запытаў.
- **Redis** — выкарыстоўваецца як агульны кэш-слой за інтэрфейсам у памяці LRU-кэша ў сэрвісе лакалізацыі, ствараючы двухслойны кэш, які забяспечвае хуткасць частых шматмоўных запытаў нават пры наяўнасці накладных выдаткаў на шыфраванне.
- **Uber Zap + Lumberjack** — абраны для структураванага лагавання з мінімальнымі выдаткамі на выдзяленне рэсурсаў і ўбудаванай ротацыяй, што дазваляе захаваць назіральнасць ядра ў прадукцыйным асяроддзі без некантраляванага росту лаг-файлаў.
- **golang-jwt / JWT** — абраны як механізм бясервернай аўтэнтыфікацыі; падпісаны токен перадаецца ў полі jwt кожнага кантэйнера /do, што забяспечвае адзіны падыход да аўтэнтыфікацыі для ўсіх кліентаў.
- **Angular 19 (+ Material, RxJS)** — абраны для стварэння рэактыўнага, кампанентна-арыентаванага вэб-кліента з гатовай сістэмай дызайну Material.
- **Tauri 2.0 + Rust + Angular** — абраны для стварэння натыўнай дэсктопнай абалонкі з мінімальнымі выдаткамі на рэсурсы шляхам паўторнага выкарыстання інтэрфейсу Angular унутры вэб-прэзентацыі на базе Rust замест убудавання поўнага браўзера.
- **Kotlin (Android) / Swift + SwiftUI (iOS)** — абраны для забеспячэння карыстальнікаў мабільных прылад сапраўды натыўнымі кліентамі з платформа-спецыфічным інтэрфейсам замест абгорткі вэб-прэзентацыі.

- **Docker / Docker Compose (сумяшчальна з Podman)** — абраны для паўторнага кантэйнерызаванага разгортвання з убудаванымі праверкамі /health, а таксама сумяшчальнасцю з Podman, што не патрабуе выкарыстання спецыяльнага дэмана ці вендара.
- **Testify (Go); Cypress/Playwright/Karma+Jasmine (кліенты)** — абраны для шматслойнага аўтаматызаванага тэсціравання, якое ахоплівае кантракт бэкэнда і інтэрфейсы кліентаў асобна, што адпавядае архітэктуры «адзін бэкэнд — шмат кліентаў».

Стан і адкрытасць інфармацыі

- **Стан: бэта.** HelixTrack Core — працоўны REST API мікрасэрвіс; пашырэнне **Documents V2** апісаны як завершаны прыкладна на 95%, але мае вядомую праблему з адлюстраваннем палёў у базе даных, таму не падаецца як цалкам гатовы прадукт.
- **Ліцэнзія: будзе вызначана.** У файле CLAUDE.md пазначана ліцэнзія MIT, але афіцыйны файл core/LICENSE змяшчае ліцэнзію Apache 2.0 — гэтае супярэчанне павінна быць вырашана да канчатковага вызначэння ліцэнзіі.
- Паказчыкі прадукцыйнасці, прыведзеныя ў README праекта (напрыклад, 50 000+ запытаў у секунду, затрымкі менш за мілісекунду), з’яўляюцца мэтавымі паказчыкамі для дызайну і маркетынгу, а не незалежна пацверджанымі вынікамі бенчмаркаў, таму яны не ўключаны ў вышэйзгаданыя сцвярджэнні.
- Кліенцкія дадаткі (Web, Desktop, Android, iOS, Aurora, HarmonyOS) захоўваюцца ў **прыватных** рэпазіторыях і апісваюцца толькі на ўзроўні прадукту.

Змест

Прыярытэтны ўзровень: Helix-першасны і флагман лінейкі прадуктаў Helix-Track — мае перавагу перад любымі праектамі Server Factory.